

MNIST from Scratch

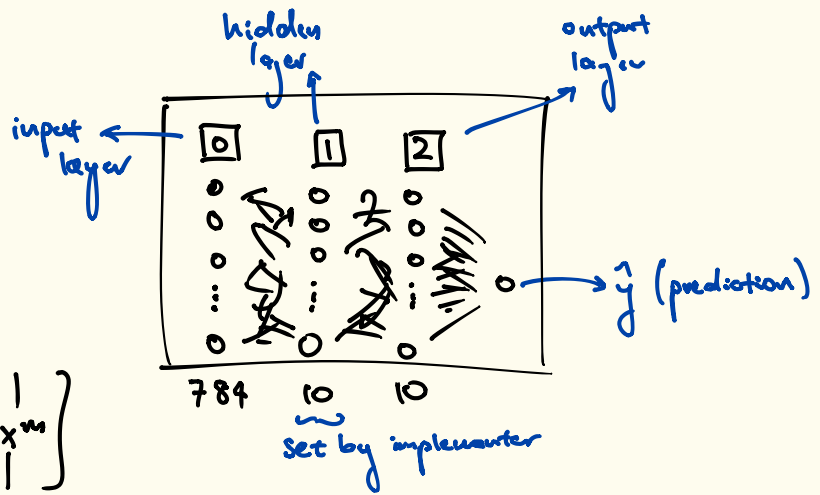
- Training images 28×28

784

Values from 0 to 255
 ↓ ↓
 black white

28 x 28

$$X = \begin{bmatrix} \text{---} & x^{(1)} & \text{---} \\ \text{---} & x^{(2)} & \text{---} \\ \vdots & \vdots & \vdots \\ \text{---} & x^{(n)} & \text{---} \end{bmatrix}^T = \begin{bmatrix} | & | & & | \\ x^1 & x^2 & \dots & x^n \\ | & | & & | \end{bmatrix}$$



Forward Propagation

① $A^{[0]} = X$ } Assume feeding 1 ing at a time
 784×1 784×1

Diagram illustrating the components of a linear equation: $y = wx + b$. The term w is labeled "weights" and the term b is labeled "bias".

(2) $Z^{[1]} = W^{[1]} A^{[0]} + B^{[1]}$ } is of linear form; regression
 $10 \times 1 \quad 10 \times 784 \quad 784 \times 1 \quad 10 \times 1$

③ $A^{[1]} = \sigma(Z^{[1]}) = \text{ReLU}(Z^{[1]})$

10×1 10×1 10×1

Activation func. used method of activation

$$\textcircled{4} \quad Z^{[2]} = W^{[2]} A^{[1]} + B^{[2]}$$

$10 \times 1 \quad 10 \times 10 \quad 10 \times 1 \quad 10 \times 1$

⑤ $A^{[2]} = \text{softmax}(Z^{[2]})$

eg Softmax (Y) =

0.008
0
0.035
0.001
0.009
0.708
0.236
0
0.008

→ # 6 has highest prob. so $\hat{y} = 6$

Softmax (Y)

Softmax Dist.

Say $Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ then $\begin{bmatrix} \frac{e^{y_1}}{\sum e^{y_i}} \\ \frac{e^{y_2}}{\sum e^{y_i}} \\ \vdots \\ \frac{e^{y_n}}{\sum e^{y_i}} \end{bmatrix}$

Softmax (Y) =

was highest prob. so $\hat{y} = 6$

In our case,

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{10} \end{bmatrix} \text{ from } 0 \text{ to } 9.$$

Softmax will output probability dist. as to which y_n has the highest prob. of being right!

Backward Propagation

- Changing weights and biases so we make better prediction in next round of forward propagation (tweaking dials)
- Start initially w/ random values
- need to optimize $w^{[1]}$, $B^{[1]}$, $w^{[2]}$, $B^{[2]}$

One-hot encoding

$$Y = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \rightarrow 3 \text{ is } 100\% \text{ chance}$$

e.g: 3 was passed

$$\hat{Y} = \begin{bmatrix} 0.001 \\ 0.2 \\ 0.702 \\ \vdots \end{bmatrix} \rightarrow 70.2\%$$

after back prop, we run forward prop again and continue the cycle.

$$\textcircled{1} \quad dZ^{[2]} = A^{[2]} - Y$$

$10 \times m \quad 10 \times m \quad 10 \times m$

→ error

$$\textcircled{2} \quad dW^{[2]} = \left(\frac{dZ^{[2]} A^{[1]T}}{m} \right)$$

$10 \times 10 \quad 10 \times m \quad m \times 10$

$$dB^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

$$\textcircled{3} \quad dZ^{[1]} = W^{[2]T} dZ^{[2]} \cdot g'(Z^{[1]})$$

$10 \times m \quad 10 \times 10 \quad 10 \times m \quad 10 \times m$

$$\textcircled{4} \quad dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$10 \times 784 \quad 10 \times m \quad m \times 784$

$$dB^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

$10 \times 1 \quad 10 \times 1$

logic is to undo forward prop. and find how much each weight/bias contributed to the error

Then we update parameters

$$\textcircled{5} \quad \begin{aligned} W^{[1]} &= W^{[1]} - \alpha dW^{[1]} \\ B^{[1]} &= B^{[1]} - \alpha dB^{[1]} \\ W^{[2]} &= W^{[2]} - \alpha dW^{[2]} \\ B^{[2]} &= B^{[2]} - \alpha dB^{[2]} \end{aligned}$$

and change it by that amount

α is learning rate (hyperparameter set by implementer)